

CS3106L - Graded Lab 4

Fault-Driven Heap Allocation in Luit

Department of Computer Science and Engineering, Indian Institute of Technology Guwahati

Contents

1. Purpose of this lab	2
2. What you should know before starting	2
3. Start from this checkpoint	3
4. Files that you may modify	3
5. The address-space rule used in this lab	4
6. Task K1: make <code>sbrk()</code> reserve memory lazily	4
6.1 Positive growth	5
6.2 Zero growth	5
6.3 Shrinking	5
6.4 What K1 should not do	5
7. Task K2: materialize one legitimate lazy page	6
7.1 Validate before allocating	6
7.2 Page-align the address	6
7.3 Reject an already valid leaf	6
7.4 Allocate, zero, and map	6
7.5 Return convention	7
8. Task K3: repair a user load/store page fault	7
9. Task K4: make kernel/user copying lazy-aware	8
9.1 Example: <code>read()</code> into a fresh heap buffer	8
9.2 Example: <code>write()</code> from a fresh heap buffer	8
9.3 <code>copyinstr()</code> also crosses the same boundary	8
9.4 The current page-table restriction	8
9.5 Permission checks remain mandatory	9
10. Behavior that is already provided	9
11. Public functional test	9
12. What the public tests are checking	10

13. Hidden tests	10
14. Grading	11
15. Recommended development order	11
16. Useful commands while debugging	11
17. Reading failures as diagnostics	12
18. Common mistakes to avoid	12
19. Submission	13
20. Final checklist	13
BrahmaputraOS / kernel Luit	

1. Purpose of this lab

In the current Luit checkpoint, `sbrk()` grows a process by allocating physical pages immediately. If a program asks for sixteen pages, the kernel allocates sixteen physical pages before the program has used a single byte of them. This is simple, but it is not how a demand-driven virtual-memory system should behave.

In this lab you will change the heap so that `sbrk()` first reserves virtual address space and physical memory is supplied only when the process actually needs a page. A load or store to an untouched heap page will therefore cause a RISC-V page fault. The kernel will examine the faulting address, decide whether the address is a legitimate lazy heap address, allocate one zero-filled page, install the PTE, and return to user mode. The same user instruction will then run again and complete normally.

There is a second part that is just as important. A process does not access user memory only through user-mode load and store instructions. The kernel also accesses user buffers on behalf of system calls. For example, `read(fd, buf, n)` eventually makes the kernel copy data into `buf`, while `write(fd, buf, n)` makes the kernel copy data out of `buf`. If `buf` belongs to a valid lazy heap region but has never been touched by user code, there may be no PTE yet. A correct lazy-allocation design must handle this case without weakening the kernel's protection against bad user pointers.

The lab therefore asks you to connect four parts of the system that you have already studied: `sbrk()`, the process address-space metadata, the RISC-V page-fault path, and the `copyout()` / `copyin()` / `copyinstr()` boundary between kernel and user memory.

You do **not** need to add or modify locks for this lab. The lock material introduced in the most recent lecture will be used in a later lab after you have had time to practise it.

2. What you should know before starting

You should be comfortable with the following ideas from the page-table and page-fault lectures:

- a virtual address may be part of a process's logical address space even when there is currently no valid leaf PTE for that address;
- `PTE_V = 0` by itself does not tell the kernel whether the address is legal or illegal;
- `scause` identifies the kind of trap and `stval` gives the faulting virtual address for a page fault;

- after a page fault is repaired, `sret` returns to the saved `sepc`, so the faulting instruction is retried;
- user pointers must never be dereferenced directly by the kernel;
- `copyout()`, `copyin()`, and `copyinstr()` are the controlled paths used when the kernel crosses the user/kernel memory boundary;
- a stack guard page must remain inaccessible even though it is an unmapped user-space page.

It is worth rereading the page-fault lecture notes before writing code. In particular, be certain that you understand the difference between an invalid PTE that represents a legitimate lazy heap page and an invalid PTE that represents an illegal address.

3. Start from this checkpoint

Use the Lab 4 package as a fresh checkpoint. Do not replace it with an older Lab 3 directory and do not copy an older full kernel tree over it. The Lab 4 package already contains the completed dependencies from the earlier labs, the corrected multi-hart boot logic, and the timing-stable QEMU grading harness.

After extracting the ZIP, enter the `LUIT_Lab04_Student` directory and run:

```
make lab04-check
make clean
make
make grade
```

The first command checks that the four graded TODO regions are still present. The normal `make grade` command runs the earlier-course regression suite. The starter checkpoint is expected to preserve those earlier tests.

Then run:

```
make grade LAB=4
```

The unmodified starter is **supposed to fail the Lab 4 functional test**. This is intentional. The starter still has eager heap allocation and the old page-fault policy so that the kernel remains usable while you work.

4. Files that you may modify

Your graded work is restricted to four marked regions.

Task	File	Region
K1	<code>kernel/proc.c</code>	TODO-BEGIN K1 ... TODO-END K1
K2	<code>kernel/vm.c</code>	TODO-BEGIN K2 ... TODO-END K2
K3	<code>kernel/trap.c</code>	TODO-BEGIN K3 ... TODO-END K3
K4	<code>kernel/vm.c</code>	TODO-BEGIN K4 ... TODO-END K4

Keep every line of code required for your solution inside these four regions. Do not move or rename the markers.

The official grader does not trust changes to the rest of the source tree. It extracts K1-K4 from your submitted files and places those regions into a clean staff checkpoint. This means that changing `user/lazytest.c`, the grading scripts, `kernel/defs.h`, `kernel/exec.c`, or any other protected file cannot help the official grade. It may only make your local tree harder to debug.

Before submitting, always run:

```
make lab04-check
```

If this command reports that a marker is missing, duplicated, or changed, fix that problem before doing anything else.

5. The address-space rule used in this lab

The checkpoint supplies one additional process field:

```
uint64 heapbase;
```

You do not implement or modify this field. The checkpoint maintains it for you.

For an `exec()`-created process, the layout around the stack and heap is conceptually:

lower addresses

```
program text / data
```

```
-----  
guard page                unmapped and invalid for ordinary access  
-----  
user stack                mapped R/W/U  
----- p->heapbase  
lazy/eager heap area  
...  
----- p->sz    (current process break)
```

higher addresses

The interval managed by `sbrk()` is therefore:

```
[p->heapbase, p->sz)
```

This interval is the key to the lab. A missing PTE is eligible for lazy materialization only if the faulting address belongs to this interval. The stack guard page is below `p->heapbase`, so it must not become valid merely because it also has no PTE.

The fixed Lab 3 shared information page is near the top of the user range at `USYSINFO_VA`. Ordinary heap growth must not reach or cross this mapping.

6. Task K1: make `sbrk()` reserve memory lazily

Open:

```
kernel/proc.c
```

Find `TODO-BEGIN K1` inside `growproc(int n)`.

The starter body is the old eager implementation. When `n > 0`, it calls `palloc()`, zeroes pages, and installs PTEs before updating `p->sz`. Your first task is to change this behavior.

6.1 Positive growth

For a positive `n`, `growproc()` should change the logical process break but should not allocate physical data pages and should not create the leaf PTEs for the new heap range.

Conceptually, the new behavior is:

```
old break = p->sz
    |
    | validate the requested growth
    v
new break = old break + n
    |
    | update p->sz
    v
return success
```

No `palloc()`.

No zero-filled data page yet.

No new heap leaf PTE yet.

You must reject arithmetic overflow and growth into the fixed `USYSINFO_VA` area.

The system-call wrapper `sys_sbrk()` already returns the old value of `p->sz`. Do not change the system-call ABI.

6.2 Zero growth

`sbrk(0)` should succeed without changing the address space. Programs often use this operation to learn the current process break.

6.3 Shrinking

For a negative `n`, the process break moves downward. The heap must never shrink below `p->heapbase`.

When complete pages leave the heap, their mappings should be removed and their physical data pages should be returned. Some pages in the range may never have been materialized. Those missing PTEs are legitimate lazy holes and must not cause a panic.

The supplied `uvmunmap()` already tolerates a missing leaf PTE. Read it before writing K1.

Remember that the process break is byte-granular while the hardware page table is page-granular. If a shrink ends in the middle of a page, that page remains mapped because some bytes in the page still belong to the heap. The public and hidden tests do not require impossible sub-page page-table protection.

6.4 What K1 should not do

Do not allocate pages in the positive-growth path. Do not move `heapbase`. Do not modify the stack or guard page. Do not change `sys_sbrk()`.

If the first public test still says that `sbrk()` consumed physical pages at reservation time, K1 is still behaving eagerly.

7. Task K2: materialize one legitimate lazy page

Open:

`kernel/vm.c`

Find:

```
int lazy_alloc_page(struct proc *p, uint64 va)
```

and the K2 region.

This helper is the single place in Lab 4 that turns one valid lazy heap page into a real mapped page.

7.1 Validate before allocating

The helper receives the original virtual address that caused a fault or was encountered by a kernel-copy operation. It must first determine whether that address is eligible for lazy allocation.

A legal address must belong to:

```
p->heapbase <= va < p->sz
```

It must also remain in the ordinary user range. An address at or above the user limit is not a lazy heap address.

Do not use the rule “PTE is invalid, therefore allocate”. That rule would incorrectly map the stack guard page, arbitrary holes, null-region addresses, and addresses beyond the process break.

7.2 Page-align the address

Physical memory is allocated one page at a time, so use the page containing `va`:

```
PGROUNDDOWN(va)
```

The mapping should cover exactly one page.

7.3 Reject an already valid leaf

The helper is intended to repair a missing lazy page. If the target already has a valid PTE, K2 must not silently replace that mapping. A page fault on a valid leaf may be a permission fault, which is a different problem and must remain an error in this lab.

7.4 Allocate, zero, and map

For a valid lazy page:

1. obtain one physical page with `palloc()`;
2. zero the complete page before exposing it to user mode;
3. map it at the page-aligned virtual address;
4. use user read/write permissions, not execute permission;
5. if mapping fails, return the physical page to the allocator;
6. make the page-table update visible before user execution resumes.

The resulting leaf should be suitable for ordinary heap data. It must not be executable.

Zero-filling is important. A newly allocated heap page must not reveal data that belonged to a previous process or a previous use of the same physical page.

7.5 Return convention

K2 returns success only when it has installed a new page. Invalid addresses, already-mapped addresses, allocation failure, and mapping failure must return an error.

8. Task K3: repair a user load/store page fault

Open:

kernel/trap.c

Find the K3 region in `usertrap()`.

RISC-V reports a user load page fault with `scause == 13` and a user store page fault with `scause == 15`. The checkpoint has already identified those two cases for you.

The faulting virtual address is available through:

`r_stval()`

Your K3 code should ask K2 whether this address is a valid lazy heap address that can be materialized.

If K2 succeeds, do not kill the process and do not advance `epc`. Allow `usertrap()` to proceed to the normal return path. When `sret` returns to user mode, the saved `sepc` still points at the same load or store instruction. That instruction is retried, but the PTE now exists, so the access can complete.

The intended successful flow is:

```
user load/store
  |
  v
MMU finds missing PTE
  |
  v
scause = 13 or 15
stval = faulting VA
  |
  v
usertrap()
  |
  v
lazy_alloc_page()
  |
  v
allocate + zero + map
  |
  v
usertrapret() / sret
  |
  v
same user instruction retries
```

If K2 rejects the address or allocation fails, preserve the existing behavior: print the diagnostic and kill only the offending process.

Do not handle instruction page faults as lazy heap faults. Heap pages in this lab are not executable.

9. Task K4: make kernel/user copying lazy-aware

Open `kernel/vm.c` again and find the K4 region inside:

```
user_resolve_pa(...)
```

The checkpoint routes `copyout()`, `copyin()`, and `copyinstr()` through this helper.

This task is necessary because not every first access to a heap page occurs in user mode.

9.1 Example: `read()` into a fresh heap buffer

Consider:

```
char *buf = sbrk(4096);  
read(fd, buf, 100);
```

After a correct K1, `buf` is a legitimate virtual address but there is still no data page behind it. The process has not executed a user store to `buf`, so no user-mode store page fault has occurred.

The kernel eventually reaches `copyout()` because it must copy file data into the user's buffer. If `copyout()` simply rejects every missing PTE, the `read()` fails even though `buf` is a legal heap address.

K4 must therefore distinguish:

missing PTE + valid current-process lazy heap address
from:

missing PTE + invalid user pointer

Only the first case may be repaired.

9.2 Example: `write()` from a fresh heap buffer

Now consider:

```
char *buf = sbrk(4096);  
write(fd, buf, 100);
```

Logically, a fresh anonymous heap page contains zeros. If user code first loaded from `buf`, K3 would materialize a zero-filled page. When the kernel itself first reads the same valid lazy buffer through `copyin()`, it should observe the same semantics. K4 therefore applies to kernel reads as well as kernel writes.

9.3 `copyinstr()` also crosses the same boundary

Pathnames and other user strings are copied with `copyinstr()`. The same pointer-validity rule applies. A valid untouched lazy page can be materialized; an arbitrary unmapped pointer must still be rejected.

9.4 The current page-table restriction

There is an important detail in this checkpoint. `copyout()` is also used by `exec()` while `exec()` is constructing a new page table that has not yet become the current process page table.

Do not use lazy heap metadata to repair arbitrary page-table objects. K4 may invoke lazy materialization only when the page table being examined is the current process's own `p->pagetable`.

For a different page table, such as the temporary page table inside `exec()`, preserve the normal strict mapping checks.

9.5 Permission checks remain mandatory

K4 must not weaken existing protection.

For all three copy paths, require a valid user mapping. For a kernel-to-user write (`copyout()`), the leaf must also be writable. This is why the supplied helper receives a `need_write` argument.

A read-only mapping such as the Lab 3 shared information page must remain read-only from the kernel-copy policy when the operation is trying to write through a user pointer.

10. Behavior that is already provided

Several pieces required by a complete lazy address space are already present in the checkpoint. You should read them, but you should not modify them for this lab.

`uvmunmap()` already skips missing PTEs. This is what allows a partially materialized heap to be shrunk safely.

`uvmcopy()` already skips missing PTEs while copying a process during `fork()`. A child therefore inherits the same logical heap size while lazy holes remain holes. Mapped pages are still copied eagerly in this lab. Copy-on-write is a later exercise.

`exec()` and `userinit()` already set `heapbase`, and `fork()` already copies it to the child.

The Lab 2 tracing and Lab 3 shared-information-page functionality are completed dependencies in this checkpoint. They are not part of Lab 4.

11. Public functional test

The public test program is:

`user/lazytest.c`

You may read it. Do not modify it for submission. The official grader uses a clean copy.

Run it through the grading harness with:

```
make grade LAB=4
```

A complete implementation should produce a Lab 4 section equivalent to:

```
--- lab 4 public test ---
[PASS] lab04 lazy memory CPUS=1
[PASS] lab04 lazy memory CPUS=4
=== Lab 4: 2/2 checks passed ===
```

If you boot Luit manually and run `lazytest`, the detailed output should end with:

```
=== Lab 4 public lazy-memory test ===
[PASS] sbrk reserves virtual memory without eager data pages
```

[PASS] first user load/store materializes zero-filled pages
[PASS] sparse heap touches allocate only on demand
[PASS] address at or beyond the process break is rejected
[PASS] stack guard page remains protected
[PASS] kernel copyout materializes a cross-page lazy destination
[PASS] kernel copyin reads an untouched lazy page as zeros
[PASS] kernel copyinstr can read a valid untouched lazy page
[PASS] shrinking frees mapped pages and tolerates lazy holes
[PASS] shrink then regrow returns fresh zero-filled memory
[PASS] fork preserves lazy holes and process isolation
LAB4: PASS

Two of the public checks deliberately cause a child process to access an illegal address. You may therefore see a kernel diagnostic such as a load/store page-fault message followed by the corresponding [PASS] line. That diagnostic is expected for the negative test. The important question is whether the bad child is killed while the kernel and the parent test process remain healthy.

12. What the public tests are checking

The first test compares the free-page count before and after reserving several heap pages. A correct lazy `sbrk()` should not consume one physical data page for every page reserved.

The next tests touch only selected pages and check that newly materialized memory begins at zero and preserves data after ordinary reads and writes.

The protection tests verify that K2 does not become an all-purpose “map any fault” routine. An address at the process break is outside `[heapbase, sz)`, and the stack guard page is below `heapbase`. Both must remain invalid.

The kernel-copy tests cover a destination that crosses a page boundary, an untouched source page that should logically contain zeros, and a user string path copied from an untouched valid lazy page.

The shrink tests verify that mapped pages are returned, unmapped lazy holes do not cause a panic, and a page that is removed and later regrown does not expose stale data.

The fork test verifies that mapped pages are copied, holes remain valid lazy holes, and the parent and child still receive independent memory when they later materialize the same logical heap page.

13. Hidden tests

The official grader includes additional cases. The exact test values are not published, but the properties are not secret. Hidden tests may use different positive and negative `sbrk()` sizes, non-page-aligned growth, page-boundary accesses, sparse heaps, cross-page kernel copies, invalid user addresses, repeated shrink/regrow cycles, allocation failure, `fork()` with different patterns of holes, and both one-hart and multi-hart QEMU configurations.

A solution should therefore implement the stated invariants rather than special-case the public values.

The hidden tests also use a clean checkpoint. Changes outside K1-K4 are not part of your graded solution.

14. Grading

The lab is graded automatically from the submitted TODO regions. The internal 100-mark breakdown is:

Component	Marks
Submission structure and extractable	5
K1-K4 regions	
Earlier Luit regression suite remains correct	10
K1: lazy <code>sbrk()</code> growth and safe shrinking	20
K2: address validation, zero-fill, mapping, failure handling	20
K3: correct user page-fault repair and retry behavior	15
K4: lazy-aware <code>copyout()</code> / <code>copyin()</code> / <code>copyinstr()</code> with protection preserved	20
Integration and hidden boundary cases	10
Total	100

Passing the public tests is necessary but does not guarantee full marks.

15. Recommended development order

Work in a sequence that leaves you with useful diagnostics after each step.

First implement K1 and run the public test. The reservation test should begin to pass, but user accesses to untouched pages may now be killed because K2 and K3 are not complete yet. That is a normal intermediate state.

Next implement K2, then K3. At that point ordinary user loads and stores to lazy heap pages should work.

Implement K4 after the user-fault path is stable. Test `read()` into untouched heap memory and `write()` from untouched heap memory. Do not weaken the bad-pointer checks simply to make these cases pass.

Finally run all regression and Lab 4 tests from a clean build.

16. Useful commands while debugging

A normal clean cycle is:

```
make lab04-check
make clean
make
make grade
make grade LAB=4
```

To boot manually:

```
make qemu
```

At the Luit shell:

```
luit$ lazytest
```

To leave QEMU when running under `-nographic`, use the standard QEMU escape sequence `Ctrl-A`, then `X`.

The grading harness is event-driven. It waits for the real `luit$` prompt and for the test's own completion marker; it does not assume that every computer boots in five or six seconds. If your machine is unusually slow, you can increase only the safety ceiling without modifying any test file:

```
LUIT_BOOT_TIMEOUT=180 make grade LAB=4
```

The package also handles the case where OpenSBI chooses a boot hart other than hart 0.

17. Reading failures as diagnostics

The public test names are intended to help you locate the problem.

If `sbrk reserves virtual memory without eager data pages` fails, inspect K1 first. A positive growth request is still consuming physical data pages.

If reservation passes but `first user load/store materializes zero-filled pages` fails and you see the process being killed by a load/store page fault, inspect K2 and K3. Check the `[heapbase, sz)` validation and the success path in `usertrap()`.

If `address at or beyond the process break is rejected` fails, K2 is probably too permissive. Do not repair every missing PTE.

If `stack guard page remains protected` fails, the same problem is more serious: the lazy allocator is turning an intentional protection hole into normal heap memory.

If the ordinary fault tests pass but `kernel copyout...`, `kernel copyin...`, or `kernel copyinstr...` fails, inspect K4. Remember that a missing PTE can be repaired only for the current process page table and only when K2 accepts the address.

If shrinking fails, recheck the negative K1 path, especially the `heapbase` lower bound and page rounding. Missing PTEs in the released range are valid.

If shrink/regrow returns old data, a materialized page was not removed correctly or the new page was not zero-filled.

If the fork test fails, first verify that you did not modify `uvmcopy()`. The supplied `uvmcopy()` already skips holes. Then check that K1 maintains `p->sz` correctly and that K2 uses the child process's copied `heapbase` and `sz` when the child later faults.

18. Common mistakes to avoid

Do not allocate the entire reserved range when the first fault occurs. A page fault should materialize one page, not all pages between `heapbase` and `sz`.

Do not use `va < p->sz` as the only validity test. That would include addresses below the heap and could destroy the stack guard-page protection. The lower bound matters.

Do not advance `epc` after a successfully repaired page fault. The faulting instruction has not completed yet; it must be retried.

Do not map heap pages executable.

Do not return success after `palloc()` or `mappages()` failure.

Do not forget to free a page that you allocated if installing its mapping fails.

Do not make `copyout()` accept a read-only PTE.

Do not use `myproc()` metadata to repair a page table that is not the current process page table.

Do not modify `uvmcopy()` to allocate holes during `fork()`. A lazy hole should remain a hole in the child until that process touches it.

Do not add a lock implementation for this lab. No new lock is required by the assignment.

19. Submission

When all tests pass, run one final clean validation:

```
make lab04-check
make clean
make
make grade
make grade LAB=4
```

Then create the submission ZIP with:

```
python3 tools/package_lab04.py YOUR_ROLL_NUMBER
```

For example:

```
python3 tools/package_lab04.py 240101001
```

The script first runs the marker check and then creates a ZIP containing exactly:

```
YOUR_ROLL_NUMBER_Lab04/
    kernel/proc.c
    kernel/vm.c
    kernel/trap.c
```

It reopens the ZIP and verifies the member list before reporting success.

Submit **only the ZIP produced by this script**. Do not manually add files to it. No report, screenshot, PDF answer sheet, or full source tree is required.

Follow the course policy in `docs/LLM_POLICY.md` for the use of external assistance.

20. Final checklist

Before uploading, verify all of the following:

- `make lab04-check` prints `LAB04 CHECK: PASS;`
- a clean `make` succeeds without warnings being promoted to errors;
- `make grade` preserves the earlier baseline;
- `make grade LAB=4` ends with `=== Lab 4: 2/2 checks passed ===;`
- your required code is inside K1-K4 only;
- you did not change the TODO markers;
- the submission ZIP was created by `tools/package_lab04.py`;

- you are uploading that ZIP rather than the complete working directory.

The purpose of this lab is not simply to make an invalid PTE become valid. The important design rule is that the kernel must know **why** the PTE is missing. A missing translation inside a reserved heap interval is recoverable. A missing translation in the guard page or outside the process break is an error. The same distinction must remain true whether the first access came from a user instruction or from the kernel while servicing a system call.